

ПОЛНЫЙ КУРС ДЛЯ НАЧИНАЮЩИХ

ОСНОВЫ программирования на Python

С нуля до собственных программ и игр. Полный годовой курс для тех, кто ещё ни разу не писал код.

5 частей · 30 уроков · 4 проекта-игры

```
print("Привет! Я начинаю учить Python.")
```

9 класс · субботний курс · Эсхата Академия

Весь путь за год

0 Часть 0 · Как думает программа

0.1. Первые команды. print и кавычки

0.2. Переменные: ящики для данных

1 Часть 1 · Первые программы

1. Разговор с программой — input

2. Числа и калькулятор

3. Условия: программа решает

4. 🎮 Проект «Угадай число»

5. Циклы: повторяем действия

6. Строки: работаем с текстом

7. 🎮 Проект «Виселица»

2 Часть 2 · Структуры и функции

8. Списки: много данных

9. Списки и циклы вместе

10. Функции: наводим порядок

11. Функции возвращают результат

12. Словари: ключ и значение

13. 🎮 «Крестики-нолики»: поле

14. 🎮 «Крестики-нолики»: игра

3 Часть 3 · Графика и большой проект

15. Черепашка turtle: линии

16. Циклы и turtle: узоры

17. Функции и turtle: имя

18. Файлы: программа запоминает

19. 📖 «Записная книжка»

20. Защита и разбор ошибок

21. Готовимся к «Змейке»

22. 🐍 «Змейка»: начало

23. 🐍 «Змейка»: финал

4

Часть 4 · Данные и свой проект

24. Анализ текста: слова

25. Защита от ошибок: try/except

26. Повторение: код-битвы

27. Выбираем финальный проект

28–30. 🚀 Финальный проект и защита



Приложения

Шпаргалка по Python · Словарь терминов · Ответы к заданиям

ЧАСТЬ 0

Как думает программа

Два коротких урока о самом главном — как вообще работает программа.

Часть 0

ЧТО В ЭТОЙ ЧАСТИ

1. **Урок 0.1.** Первые команды. `print` и кавычки
2. **Урок 0.2.** Переменные: ящики для данных

9 класс · субботний курс · Эсхата Академия

Первые команды: программа делает то, что сказано

ЧЕМУ НАУЧИМСЯ

- Понять, что программа — это команды, которые компьютер выполняет по порядку
- Написать первую команду **print** и увидеть результат
- Разобраться, зачем нужны кавычки



Что такое программа

Программа — это **список команд для компьютера**. Компьютер читает их сверху вниз, одну за другой, и выполняет ровно то, что написано. Он не догадывается и не додумывает — делает буквально.



На пальцах. Представь, что объясняешь другу, как заварить чай: налей воду, вскипяти, положи заварку, подожди. Это команды по порядку. Если перепутать шаги — чай не получится. Программа — то же самое, только компьютер ещё строже: он не поймёт намёков, ему нужна точность.



Первая команда — print

Команда **print** означает «покажи на экране». То, что стоит в скобках и кавычках, компьютер выведет как есть:

```
privet.py

print("Привет!")
print("Меня зовут Python.")
print("Я выполняю команды по порядку.")
```

НА ЭКРАНЕ ПОЯВИТСЯ

```
Привет!
Меня зовут Python.
Я выполняю команды по порядку.
```

Три команды — три строки на экране, ровно в том порядке, как написаны. Поменяешь строки местами — изменится и порядок вывода. Вот что значит «программа = команды по порядку».



ЗАДАНИЕ В ПАРЕ

Расскажи о себе

Напишите программу из трёх команд `print` : пусть она выведет твоё имя, твой город и твой возраст — каждое на своей строке.

Водитель печатает, штурман диктует. Каждую строку сразу запускайте клавишей F5.



Главное открытие: кавычки

А теперь фокус. Наберите две очень похожие строки и запустите:

kavychki.py

```
print("2 + 2")
print(2 + 2)
```

НА ЭКРАНЕ ПОЯВИТСЯ

```
2 + 2
4
```

ЗАПОМНИ ЭТО ПРАВИЛО

Кавычки говорят: **"покажи ровно эти значки"**. Без кавычек компьютер думает, что это задачка, и **решает** её. Кавычки = «как есть», без кавычек = «посчитай».



Частые ошибки

ТАК НЕ РАБОТАЕТ	А ТАК ПРАВИЛЬНО
<code>print(Привет)</code>	<code>print("Привет")</code>
<code>print('Привет')</code>	<code>print("Привет")</code>
<code>prin("Привет")</code>	<code>print("Привет")</code>

Компьютер не догадается, что ты имел в виду. Ошибка — это не провал, а подсказка: прочитай, что он пишет, и поправь.



ДОМАШНЕЕ ЗАДАНИЕ

1. Программа выводит твоё имя и три факта о тебе (три команды `print`).
2. Программа выводит расписание твоих уроков на завтра.

★ **Со звёздочкой:** нарисуй рамку из символов `*` и напиши внутри «Я учу Python».

Переменные: ящики для данных

ЧЕМУ НАУЧИМСЯ

- Понять, что такое переменная и зачем она нужна
- Класть значение в «ящик» и доставать его по имени
- Считать с помощью переменных



Зачем нужны переменные

На прошлом уроке мы писали всё прямо в `print`. Но что делать, если одно значение нужно использовать много раз? Каждый раз печатать заново? Нет — его можно **сохранить**.



На пальцах. Переменная — это **ящик с наклейкой**. На наклейке — имя, внутри — значение. Ты кладёшь что-то один раз, а достаёшь столько раз, сколько нужно. Захотел поменять содержимое — положил новое, старое пропало.



Кладём в ящик

Знак `=` здесь означает не «равно», как в математике, а «**положи в ящик**». Слева — имя ящика, справа — что кладём:

yaschiki.py

```
age = 15
print(age)

name = "Алишер"
print(name)
```

НА ЭКРАНЕ ПОЯВИТСЯ

```
15
Алишер
```

Мы написали имя ящика — и компьютер достал из него значение. Читается так: «ящик `age` получает значение 15».



Зачем это удобно

Один раз положил — используешь сколько угодно раз:

privet_imya.py

```
name = "Алишер"
print("Привет,")
print(name)
print("тебе лет:")
print(age)
```

И самое полезное — с числами в ящиках можно **считать**. Вот расчёт покупки:

покупка.py

```
price = 25      # сомони за штуку
count = 4       # сколько штук
print(price * count)
```

НА ЭКРАНЕ ПОЯВИТСЯ
100

ЗАПОМНИ ЭТО ПРАВИЛО

Знак `=` — это «положи в ящик», а не «равно». Пишешь имя ящика — компьютер достаёт значение. Новое значение стирает старое.

👉 ЗАДАНИЕ В ПАРЕ

Средний балл

Положите пять своих оценок в ящики `g1` ... `g5`, а потом выведите средний балл: сумму разделите на 5. Подсказка: `print((g1 + g2 + g3 + g4 + g5) / 5)`

Через 10 минут поменяйтесь ролями — теперь печатает штурман.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
print(Name)
```

```
age = "15"
print(age + 1)
```

```
print(age)
age = 15
```

А ТАК ПРАВИЛЬНО

```
print(name)
```

```
age = 15
print(age + 1)
```

```
age = 15
print(age)
```

Python различает большие и маленькие буквы: `name` и `Name` — разные ящики. И сначала клади в ящик, потом доставай — не наоборот.

ДОМАШНЕЕ ЗАДАНИЕ

1. Создай ящики для профиля: ник, город, любимый предмет. Выведи их красиво.
2. Есть цена товара и количество — посчитай общую стоимость в сомони.

★ **Со звёздочкой:** положи в ящики цены трёх покупок и выведи, сколько всего денег нужно.

Первые программы

Ввод, числа, условия, циклы и строки. И две первые настоящие игры, которые ты напишешь сам.

Четверть 1 · 7 уроков

ЧТО В ЭТОЙ ЧАСТИ

1. **Урок 1.** Разговор с программой — input
2. **Урок 2.** Числа и калькулятор
3. **Урок 3.** Условия: программа решает
4. **Урок 4.** 🎮 Проект «Угадай число»
5. **Урок 5.** Циклы: повторяем действия
6. **Урок 6.** Строки: работаем с текстом
7. **Урок 7.** 🎮 Проект «Виселица»

Разговор с программой: команда `input`

ЧЕМУ НАУЧИМСЯ

- Спрашивать у пользователя данные командой `input`
- Сохранять ответ в переменную и использовать его
- Написать программу, которая знакомится с человеком



Программа умеет спрашивать

Раньше все данные мы записывали в код сами. Но настоящая программа **общается**: спрашивает у человека и получает ответ. За это отвечает команда `input` — «спроси и подожди ответ».



На пальцах. `input` — это как продавец, который спрашивает «что вам?» и ждёт. Пока ты не ответишь, программа стоит на месте. Ответ она кладёт в ящик-переменную, чтобы использовать дальше.



Разбираем код

Программа спрашивает имя и город, потом здоровается:

znakomstvo.py

```
name = input("Как тебя зовут? ")
city = input("Из какого ты города? ")

print("Привет, " + name + "!")
print("Значит, ты из города " + city + ". Отличное место!")
```

ПРИМЕР РАБОТЫ

```
Как тебя зовут? Алишер
Из какого ты города? Худжанд
Привет, Алишер!
Значит, ты из города Худжанд. Отличное место!
```

Знак `+` здесь **склеивает** текст: строку «Привет, », ответ из ящика `name` и «!». Это называется соединением строк.

ВАЖНО ЗАПОМНИТЬ

input **всегда возвращает текст**, даже если ввели число. «15» после input — это текст, а не число. Как превратить его в число — узнаем на следующем уроке.

ЗАДАНИЕ В ПАРЕ

Анкета друга

Программа спрашивает имя, любимую еду и любимую игру, а потом составляет фразу: « **имя** любит **еда** и играет в **игра** ».

Водитель печатает, штурман диктует. Через 10 минут — меняетесь.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
name = input(Как зовут?)
```

```
print("Привет " name)
```

А ТАК ПРАВИЛЬНО

```
name = input("Как зовут? ")
```

```
print("Привет " + name)
```

Вопрос внутри input тоже в кавычках. А чтобы приклеить переменную к тексту — нужен знак **+**.

ДОМАШНЕЕ ЗАДАНИЕ

1. Программа спрашивает имя и класс, затем печатает: « **имя** учится в **класс** классе ».
2. Программа спрашивает город и отвечает: «Я бы хотел побывать в **город** !»

★ **Со звёздочкой:** спроси имя друга и «нарисуй» ему открытку из символов с его именем внутри.

Числа и калькулятор

ЧЕМУ НАУЧИМСЯ

- Превращать текст в число командой `int`
- Считать: сложение, вычитание, умножение, деление, остаток
- Написать калькулятор из двух чисел



Почему `input` мало

На прошлом уроке мы узнали: `input` даёт **текст**. С текстом нельзя считать — компьютер не станет складывать «2» и «3» как числа, он их склеит в «23». Чтобы считать, текст нужно превратить в число командой `int`.



На пальцах. «5» в кавычках — это как цифра, нарисованная на бумаге: красивая, но сложить нельзя. `int` превращает нарисованную цифру в настоящее число, с которым уже можно работать.



Разбираем код

Калькулятор: спрашиваем два числа и показываем все действия:

```
calc.py

a = int(input("Введи первое число: "))
b = int(input("Введи второе число: "))

print("Сумма:", a + b)
print("Разность:", a - b)
print("Произведение:", a * b)
print("Деление:", a / b)
print("Остаток:", a % b)
```

ЕСЛИ ВВЕСТИ 10 И 3

Сумма: 13
Разность: 7
Произведение: 30
Деление: 3.33...
Остаток: 1

Обрати внимание: `int(input(...))` — это две команды сразу. Сначала `input` спрашивает, потом `int` превращает ответ в число. А запятая в `print` ставит пробел между текстом и результатом.

ЗНАКИ ДЕЙСТВИЙ В PYTHON

`+` сложить · `-` вычесть · `*` умножить · `/` разделить · `%` остаток от деления. Остаток очень пригодится дальше — он умеет проверять чётность.

👉 ЗАДАНИЕ В ПАРЕ

Магазинный чек

Программа спрашивает цену товара и количество, потом считает и выводит общую сумму в сомони. Подсказка: не забудь `int` для обоих ответов.

Через 10 минут поменяйтесь ролями.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
a = input("Число: ")
print(a + 5)
```

```
print("Сумма:" a + b)
```

А ТАК ПРАВИЛЬНО

```
a = int(input("Число: "))
print(a + 5)
```

```
print("Сумма:", a + b)
```

Без `int` ответ остаётся текстом. И между кусочками внутри `print` нужна запятая.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Программа спрашивает возраст и считает, сколько тебе будет через 10 лет.
2. Программа переводит сумму в сомони в дирамы (1 сомони = 100 дирам).

★ **Со звёздочкой:** калькулятор чаевых — спроси счёт, добавь 10% и выведи итог.

Условия: программа принимает решения

ЧЕМУ НАУЧИМСЯ

- Учить программу выбирать действие по условию
- Разбирать **if** и **else** — «если ... иначе ...»
- Проверять число на чётность



Что такое условие

Пока наши программы выполняли все команды подряд. Но настоящая программа умеет **выбирать**: сделать одно или другое, смотря по обстоятельствам.



На пальцах. Утром смотришь в окно. **Если** дождь — берёшь зонт. **Иначе** — не берёшь. Это решение по условию. Программа делает так же: проверяет условие и выбирает путь.

В Python это слова **if** (если) и **else** (иначе). После условия — двоеточие, а команды внутри пишутся **с отступом** в 4 пробела. Отступ показывает, что команда относится к условию.



Разбираем код

Программа проверяет возраст:

```
vozrast.py

age = int(input("Сколько тебе лет? "))

if age >= 18:
    print("Ты совершеннолетний.")
else:
    print("Ты ещё несовершеннолетний.")
```

ЕСЛИ ВВЕСТИ 15

Ты ещё несовершеннолетний.

Компьютер проверил: **15 >= 18**? Неправда — значит, он пропустил первую ветку и выполнил ту, что после **else**.

А вот проверка чётности — здесь пригодился остаток **%** с прошлого урока:

```
chet.py
```

```
number = int(input("Введи число: "))
if number % 2 == 0:
    print("Число чётное.")
else:
    print("Число нечётное.")
```

ВАЖНО ЗАПОМНИТЬ

Двоеточие в конце условия. Знак `==` — это «сравнить» (а один `=` — «положить в ящик»). И отступ 4 пробела у команд внутри `if` и `else`.

👉 ЗАДАНИЕ В ПАРЕ

Больше из двух

Программа спрашивает два числа и выводит большее. Подсказка: `if a > b: ... else: ...`

Через 10 минут поменяйтесь ролями.



Частые ошибки

ТАК НЕ РАБОТАЕТ	А ТАК ПРАВИЛЬНО
<code>if age >= 18</code>	<code>if age >= 18:</code>
<code>if age = 18:</code>	<code>if age == 18:</code>
<code>if age >= 18:</code> <code>print(...)</code>	<code>if age >= 18:</code> <code>print(...)</code>

🏠 ДОМАШНЕЕ ЗАДАНИЕ

- Программа спрашивает оценку (2–5) и пишет словами: «отлично», «хорошо» и т.д. Подсказка: пригодится `elif` — «иначе если».
- Программа-«фейсконтроль»: пускает на фильм с 16 лет, иначе отказывает.

★ **Со звёздочкой:** программа проверяет, високосный ли год (делится на 4).

 «Угадай
число»

ЧТО ПОСТРОИМ

- Первую настоящую игру: компьютер загадал число, ты угадываешь
- Познакомимся с циклом **while** — «повторяй, пока...»
- Научимся считать попытки и завершать цикл словом **break**

 Что за игра

Компьютер загадывает число от 1 до 100. Ты называешь варианты, а он подсказывает: «больше» или «меньше», пока не угадаешь. В конце покажет, за сколько попыток ты справился. Это твоя **первая игра** — и она уместается в 15 строк.



Новое слово — цикл. Цикл **while** означает «повторяй, пока условие верно». Здесь мы повторяем вопрос снова и снова, пока число не угадано. Без цикла пришлось бы копировать один и тот же код сто раз.

 Собираем игру

```
guess.py

import random

secret = random.randint(1, 100)
attempts = 0

print("Я загадал число от 1 до 100!")

while True:
    guess = int(input("Твоя догадка: "))
    attempts = attempts + 1

    if guess < secret:
        print("Больше!")
    elif guess > secret:
        print("Меньше!")
    else:
        print("Верно! Попыток:", attempts)
        break
```

- 1 random загадывает число**
`random.randint(1, 100)` — компьютер выбирает случайное число и прячет его в ящик `secret`.
- 2 while True — повторяем без конца**
Цикл крутится, спрашивая догадку снова и снова, пока мы сами его не остановим.
- 3 elif — третья ветка**
Теперь веток три: меньше, больше, точно. Слово `elif` — это «иначе если».
- 4 break останавливает игру**
Когда угадали — `break` выходит из цикла, и игра заканчивается.

ЗАДАНИЕ В ПАРЕ

Соберите и сыграйте

Наберите игру и сыграйте по очереди друг против друга. Работает? Поздравляю — вы написали первую игру!

Один печатает, второй проверяет каждую строку. Потом меняетесь и играете.

ДОМАШНЕЕ ЗАДАНИЕ

- Ограничь число попыток семью. Если не угадал за 7 — программа говорит, какое было число.
- Поменяй диапазон на 1–1000 и посмотри, за сколько попыток реально угадать.

★ **Со звёздочкой:** поменяйтесь ролями с компьютером — теперь ты загадываешь, а программа угадывает.

Циклы for: повторяем по счёту

ЧЕМУ НАУЧИМСЯ

- Повторять действие заданное число раз циклом **for**
- Понимать **range** — «диапазон чисел»
- Напечатать таблицу умножения и обратный отсчёт



Два вида циклов

В «Угадай число» мы видели цикл **while** — «повторяй, пока не угадали». Но часто мы знаем **точно, сколько раз** надо повторить: 10 строк, 5 учеников, 12 месяцев. Для этого удобнее цикл **for**.



На пальцах. **while** — это «мешай суп, пока не будет готов» (не знаешь сколько). **for** — это «отожмись 10 раз» (знаешь точно). Оба повторяют, но по-разному считают.



Разбираем код

Обратный отсчёт и таблица умножения:

```
cikly.py

# обратный отсчёт от 10 до 1
for i in range(10, 0, -1):
    print(i)
print("Старт!")

# таблица умножения на 7
for i in range(1, 11):
    print(7, "x", i, "=", 7 * i)
```

НАЧАЛО ВЫВОДА

```
10
9
8
...
Старт!
7 x 1 = 7
7 x 2 = 14
...
```

Переменная `i` — это счётчик: на каждом повторе она берёт следующее число из диапазона. `range(1, 11)` даёт числа от 1 до 10 (последнее число 11 не входит — важно запомнить!).

КАК ЧИТАТЬ RANGE

`range(1, 11)` — от 1 до 10 (правая граница не входит). `range(10, 0, -1)` — от 10 до 1 в обратную сторону. Третье число — шаг.

👉 ЗАДАНИЕ В ПАРЕ

Сумма от 1 до 100

С помощью цикла `for` посчитайте сумму всех чисел от 1 до 100. Подсказка: заведите ящик `total = 0` и на каждом шаге прибавляйте к нему `i`.

Через 10 минут поменяйтесь ролями.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
for i in range(1, 10):  
    # ждём до 10
```

```
for i in range(5)
```

А ТАК ПРАВИЛЬНО

```
for i in range(1, 11):  
    # теперь до 10
```

```
for i in range(5):
```

Правая граница `range` не входит. И двоеточие в конце строки `for` — обязательно.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Выведи все чётные числа от 2 до 20.
2. Программа спрашивает число `N` и печатает таблицу умножения на него.

★ **Со звёздочкой:** посчитай факториал числа `N` ($1 \times 2 \times 3 \times \dots \times N$).

Строки: работаем с текстом

ЧЕМУ НАУЧИМСЯ

- Узнавать длину текста и брать отдельные буквы
- Менять регистр: **upper** и **lower**
- Проверять, есть ли буква в слове — это ядро игры «Виселица»



Текст — это тоже данные

Слово в Python называется **строкой**. С ним можно многое: узнать длину, вытащить любую букву, перевести в заглавные, проверить, есть ли в нём нужная буква. Всё это нам понадобится уже на следующем уроке — для игры «Виселица».



На пальцах. Строка — это как бусы из букв. Каждая буква на своём месте, у каждого места есть номер. Только счёт начинается с нуля: первая буква — номер 0.



Разбираем код

```
stroki.py

word = input("Введи слово: ")

print("Длина:", len(word))
print("Первая буква:", word[0])
print("Последняя буква:", word[-1])
print("Заглавными:", word.upper())

letter = input("Какую букву ищем? ")
if letter in word:
    print("Есть такая буква!")
else:
    print("Нет такой буквы.")
```

ЕСЛИ ВВЕСТИ «ПАМИР» И БУКВУ «М»

Длина: 5
Первая буква: п
Последняя буква: р
Заглавными: ПАМИР
Есть такая буква!

`len(word)` — длина, `word[0]` — первая буква (счёт с нуля!), `word[-1]` — последняя. А `letter in word` проверяет, есть ли буква в слове — вот это и есть главная проверка «Виселицы».

ВАЖНО ЗАПОМНИТЬ

Буквы нумеруются с нуля: у слова из 5 букв номера 0, 1, 2, 3, 4. Номер `-1` — это последняя буква. Оператор `in` отвечает «да» или «нет»: есть буква в слове или нет.

👉 ЗАДАНИЕ В ПАРЕ

Разбор имени

Программа спрашивает имя и выводит: его длину, первую и последнюю букву, имя заглавными буквами.

Через 10 минут поменяйтесь ролями.



Частые ошибки

ТАК НЕ РАБОТАЕТ	А ТАК ПРАВИЛЬНО
<code>word[1]</code> # первая буква	<code>word[0]</code> # первая буква
<code>word.upper</code>	<code>word.upper()</code>

Счёт с нуля: первая буква — `[0]`. И у методов вроде `upper` обязательны скобки в конце.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Программа считает, сколько букв «а» в введённом слове.
2. Программа выводит слово наоборот. Подсказка: `word[::-1]`.

★ **Со звёздочкой:** проверь, читается ли слово одинаково слева и справа (палиндром).



«Виселица»

ЧТО ПОСТРОИМ

- Игру, где нужно угадать слово по буквам
- Соберём вместе всё: строки, циклы, условия, списки
- Это итоговый проект четверти — с защитой перед классом



Что за игра

Компьютер загадывает слово и показывает его чёрточками: `_ _ _ _ _`. Игрок называет буквы. Угадал — буква открывается. Ошибся — минус попытка. Слов даётся 6 попыток. Откроешь слово — победа. Это твой **главный проект четверти**.



Собираем игру

```
viselica.py

import random
words = ["худжанд", "душанбе", "памир", "python"]
secret = random.choice(words)

guessed = []
mistakes = 0

while mistakes < 6:
    display = ""
    for letter in secret:
        if letter in guessed:
            display = display + letter + " "
        else:
            display = display + "_ "
    print(display)

    if all(l in guessed for l in secret):
        print("Победа! Слово:", secret)
        break

    guess = input("Назови букву: ")
    if guess in secret:
        guessed.append(guess)
    else:
```

```
mistakes = mistakes + 1
print("Ошибка:", mistakes, "из 6")
```

1 Список слов и выбор

`words` — это список (несколько слов в квадратных скобках). `random.choice` выбирает одно случайное.

2 Показываем слово чёрточками

Цикл `for` проходит по каждой букве: угадана — показываем её, нет — рисуем `_`.

3 `append` добавляет в список

Угаданную букву кладем в список `guessed` командой `append` — «добавь в конец».

4 Проверка победы

`all(...)` отвечает «да», если все буквы слова уже угаданы. Тогда — победа и `break`.



ИТОГОВЫЙ ПРОЕКТ ЧЕТВЕРТИ

Соберите, доработайте, защитите

Наберите игру в паре. Затем сделайте её своей: добавьте слова про Таджикистан, придумайте поздравление за победу. На защите (2 минуты у проектора) расскажите: что делает игра, как работает, что было сложно.

Оба в паре должны уметь объяснить любую строку кода.



ЧТО ПОКАЗАТЬ НА ЗАЩИТЕ

1. Игра запускается и работает без ошибок.
2. Вы добавили минимум 5 своих слов.
3. Каждый может объяснить, что делает любая строка.

★ **Со звёздочкой:** покажите игроку, сколько попыток осталось, и вставьте поздравление с именем победителя.

Структуры и функции

Списки хранят много данных, функции наводят порядок в коде. А в конце — игра на двоих.

Четверть 2 · 7 уроков

ЧТО В ЭТОЙ ЧАСТИ

1. **Урок 8.** Списки: много данных в одном ящике
2. **Урок 9.** Списки и циклы вместе
3. **Урок 10.** Функции: наводим порядок
4. **Урок 11.** Функции возвращают результат
5. **Урок 12.** Словари: ключ и значение
6. **Урок 13.** 🎮 Проект «Крестики-нолики»: поле
7. **Урок 14.** 🎮 «Крестики-нолики»: игра и защита

Списки: много данных в одном ящике

ЧЕМУ НАУЧИМСЯ

- Хранить много значений в одном списке
- Брать элементы по номеру и добавлять новые командой **append**
- Узнавать длину списка



Зачем нужны списки

Пока в каждый ящик-переменную мы клали **одно** значение. А если нужно хранить пять оценок? Заводить `g1`, `g2`, `g3`, `g4`, `g5` — неудобно. Есть решение получше: **список** — один ящик, куда помещается сразу много значений.



На пальцах. Переменная — это один ящик. Список — это полка с пронумерованными ячейками. Всё лежит вместе, а каждая ячейка со своим номером. Считаем с нуля: первая ячейка — номер 0.



Разбираем код

Список создаётся квадратными скобками, элементы через запятую:

```
spiski.py

fruits = ["яблоко", "банан", "дыня"]

print(fruits[0])      # яблоко — первый
print(fruits[2])      # дыня — третий
print(len(fruits))    # 3 — сколько всего

fruits.append("виноград") # добавить в конец
print(fruits)
```

НА ЭКРАНЕ ПОЯВИТСЯ

```
яблоко
дыня
3
['яблоко', 'банан', 'дыня', 'виноград']
```

Как и у строк, счёт с нуля: `fruits[0]` — первый элемент. `len` говорит, сколько всего элементов. А `append` добавляет новый в конец списка.

ВАЖНО ЗАПОМНИТЬ

Список — в квадратных скобках `[]`. Элементы нумеруются с нуля. `append` — «добавь в конец». `len(список)` — сколько элементов внутри.

ЗАДАНИЕ В ПАРЕ

Список любимого

Создайте список из трёх любимых блюд. Выведите первое и последнее (подсказка: последнее — это `[-1]`). Добавьте в список четвёртое блюдо и выведите весь список.

Водитель печатает, штурман диктует. Через 10 минут — меняетесь.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
fruits[1] # первый
```

```
fruits.append "манго"
```

А ТАК ПРАВИЛЬНО

```
fruits[0] # первый
```

```
fruits.append("манго")
```

Счёт с нуля, как у строк. И у `append` обязательны скобки с тем, что добавляем.

ДОМАШНЕЕ ЗАДАНИЕ

1. Составь список из 5 городов Таджикистана и выведи каждый.
2. Заведи пустой список `[]`, добавь в него три числа через `append`.

★ **Со звёздочкой:** список дел на день — программа спрашивает 3 дела и складывает их в список.

Списки и циклы: обрабатываем данные

ЧЕМУ НАУЧИМСЯ

- Перебирать список циклом **for**
- Считать сумму, среднее, находить максимум
- Обрабатывать оценки целого класса



Список + цикл = сила

На уроке 5 мы перебирали числа через `range`. Но `for` умеет перебирать и **список напрямую** — брать по очереди каждый элемент. Это соединяет две вещи, которые мы уже знаем, и открывает настоящую обработку данных.



На пальцах. Представь стопку тетрадей. Цикл `for` берёт по одной тетради сверху, что-то с ней делает, кладёт, берёт следующую — пока стопка не кончится. Список — это стопка, `for` — твои руки.



Разбираем код

Считаем средний балл по оценкам класса:

```
ocenki.py

grades = [5, 4, 5, 3, 4]

total = 0
for grade in grades:
    total = total + grade

print("Сумма:", total)
print("Средний балл:", total / len(grades))
print("Лучшая оценка:", max(grades))
```

НА ЭКРАНЕ ПОЯВИТСЯ

```
Сумма: 21
Средний балл: 4.2
Лучшая оценка: 5
```

Цикл берёт каждую оценку и прибавляет к `total`. После цикла делим сумму на количество — получаем среднее. А `max` сам находит наибольшее число в списке (есть и `min` — наименьшее).

ВАЖНО ЗАПОМНИТЬ

`for x in список` — берёт каждый элемент по очереди. Чтобы копить сумму, заведи ящик `total = 0` до цикла. Готовые помощники: `max`, `min`, `sum`.

👉 ЗАДАНИЕ В ПАРЕ

Магазинная корзина

Есть список цен: `[12, 45, 8, 30, 15]`. Посчитайте общую стоимость покупки и выведите самый дорогой товар. Подсказка: `sum(...)` и `max(...)`.

Через 10 минут поменяйтесь ролями.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
for grade in grades:
    total = grade
```

```
total = 0 # внутри цикла
```

А ТАК ПРАВИЛЬНО

```
for grade in grades:
    total = total + grade
```

```
total = 0 # до цикла!
```

Ящик для суммы заводится **до** цикла, иначе он обнуляется каждый раз. И прибавляй к `total`, а не перезаписывай.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

- В списке температур `[18, 22, 25, 19, 30]` найди среднюю, самую высокую и низкую.
- Посчитай, сколько в списке оценок «пятёрок» (подсказка: `if` внутри цикла + счётчик).

★ **Со звёздочкой:** выведи только те оценки из списка, которые выше среднего балла.

Функции: наводим порядок в коде

ЧЕМУ НАУЧИМСЯ

- Создавать свои команды — функции — словом **def**
- Передавать в функцию данные (аргументы)
- Не повторять один и тот же код по многу раз



Зачем нужны функции

Мы уже пользовались готовыми командами: `print`, `input`, `len`. Это **функции** — маленькие помощники, которые делают одно дело. Оказывается, свои функции можно создавать самому — и это главный способ навести порядок в коде.



На пальцах. Функция — это рецепт с названием. Один раз записал «как заварить чай», а потом просто говоришь «завари чай» — и не пересказываешь все шаги заново. Так и в коде: описал функцию раз — вызываешь её сколько угодно.



Разбираем код

Создаём свою функцию-приветствие словом **def**:

```
funkcii.py

def privet(name):
    print("Привет, " + name + "!")
    print("Рады тебя видеть.")

privet("Алишер")
privet("Фарход")
privet("Мадина")
```

НА ЭКРАНЕ ПОЯВИТСЯ

```
Привет, Алишер!
Рады тебя видеть.
Привет, Фарход!
Рады тебя видеть.
Привет, Мадина!
Рады тебя видеть.
```

Строка `def privet(name):` описывает функцию: её имя `privet` и что она принимает — `name`. Тело функции — с отступом. Дальше мы **вызываем** её три раза с разными именами — и код внутри не пришлось повторять.

ВАЖНО ЗАПОМНИТЬ

`def имя(данные):` — создаём функцию. Тело — с отступом. То, что в скобках при создании — **параметр**, то, что передаём при вызове — **аргумент**. Функцию сначала описывают, потом вызывают.

👉 ЗАДАНИЕ В ПАРЕ

Функция-рамка

Напишите функцию `ramka(text)`, которая печатает текст в рамке из звёздочек. Вызовите её с тремя разными словами.

Через 10 минут поменяйтесь ролями.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
def privet(name)
```

```
def privet(name):  
print(name)
```

А ТАК ПРАВИЛЬНО

```
def privet(name):
```

```
def privet(name):  
    print(name)
```

Двоеточие после строки `def`, тело — с отступом. И не забудь вызвать функцию — само по себе описание ничего не печатает.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Функция `privetstvie(name, city)` — здоровается и называет город.
2. Функция `ploshad(a, b)` — печатает площадь прямоугольника.

★ **Со звёздочкой:** функция рисует треугольник из звёздочек заданной высоты.

Функции возвращают результат

ЧЕМУ НАУЧИМСЯ

- Получать из функции результат словом **return**
- Понимать разницу между «напечатать» и «вернуть»
- Собирать программу из функций-помощников



Печатать или вернуть?

Функции с прошлого урока сами печатали результат. Но чаще нужно, чтобы функция **отдала** ответ обратно — и мы решили, что с ним делать: напечатать, сохранить в ящик, использовать в расчёте. За это отвечает слово **return**.



На пальцах. `print` — это когда повар выносит блюдо и сразу ставит на стол. `return` — когда повар отдаёт блюдо тебе в руки, а ты сам решаешь: съесть, отнести другу или положить в холодильник. `return` гибче.



Разбираем код

```
return.py

def summa(a, b):
    return a + b

# результат можно сохранить в ящик
result = summa(10, 5)
print(result)           # 15

# или сразу использовать в расчёте
print(summa(3, 4) * 2)  # 14
```

НА ЭКРАНЕ ПОЯВИТСЯ

```
15
14
```

Функция `summa` не печатает — она **возвращает** число. Поэтому его можно сохранить в `result`, умножить, передать дальше. В этом сила `return`: результат становится обычным значением, с которым можно делать что угодно.

ВАЖНО ЗАПОМНИТЬ

`return` отдаёт результат из функции наружу. После `return` функция заканчивает работу.
Разница: `print` показывает на экране, `return` отдаёт значение, которое можно использовать.

👉 ЗАДАНИЕ В ПАРЕ

Калькулятор на функциях

Напишите функции `summa(a,b)`, `raznost(a,b)`, `proizvedenie(a,b)` — каждая возвращает результат. Затем спросите два числа и покажите все три действия.

Через 10 минут поменяйтесь ролями.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
def summa(a, b):  
    print(a + b)  
x = summa(2, 3) # x пустой
```

А ТАК ПРАВИЛЬНО

```
def summa(a, b):  
    return a + b  
x = summa(2, 3) # x = 5
```

Если функция только печатает (`print`), в ящик ничего не попадёт. Чтобы сохранить результат — нужен `return`.



ДОМАШНЕЕ ЗАДАНИЕ

1. Функция `srednee(a, b, c)` — возвращает среднее трёх чисел.
2. Функция `max_iz_dvuh(a, b)` — возвращает большее из двух.

★ **Со звёздочкой:** функция `chetnoe(n)` возвращает `True` или `False` — чётное ли число.

Словари: ключ и значение

ЧЕМУ НАУЧИМСЯ

- Хранить пары «ключ — значение» в словаре
- Находить значение по ключу
- Построить телефонную книгу



Когда списка мало

В списке элементы ищут по номеру: `[0]`, `[1]`. Но иногда искать хочется по **имени**: не «третий элемент», а «телефон Алишера». Для этого есть **словарь** — он хранит пары: ключ (имя) и значение (телефон).



На пальцах. Словарь — как настоящий словарь или телефонная книга. Ты не листаешь по номерам страниц — ты ищешь слово и сразу видишь перевод. Ключ — это слово, значение — его перевод.



Разбираем код

Словарь создаётся фигурными скобками, пары через двоеточие:

```
slovari.py

phones = {
    "Алишер": "901-234-567",
    "Мадина": "902-345-678",
    "Фарход": "903-456-789"
}

print(phones["Мадина"])    # находим по имени

phones["Сафар"] = "904-567-890" # добавляем нового
print(phones)
```

НА ЭКРАНЕ ПОЯВИТСЯ

```
902-345-678
{'Алишер': '901-234-567', 'Мадина': '902-345-678', ...}
```

Вместо номера, как в списке, мы пишем **ключ** — `phones["Мадина"]` — и сразу получаем значение. Чтобы добавить новую пару, просто присваиваем значение новому ключу.

СПИСОК ПРОТИВ СЛОВАРЯ

Список `[]` — доступ по номеру: `spisok[0]`. Словарь `{ }` — доступ по ключу: `slovar["имя"]`. Список — когда важен порядок, словарь — когда важно найти по названию.

ЗАДАНИЕ В ПАРЕ

Словарь оценок

Создайте словарь, где ключ — предмет, значение — твоя оценка: математика, история, информатика. Выведите оценку по информатике и добавьте новый предмет.

Через 10 минут поменяйтесь ролями.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
phones = ["Алишер": "901"]
```

```
phones[0] # первый
```

А ТАК ПРАВИЛЬНО

```
phones = {"Алишер": "901"}
```

```
phones["Алишер"]
```

Словарь — в фигурных скобках `{ }`, а не квадратных. И ищем по ключу, а не по номеру.

ДОМАШНЕЕ ЗАДАНИЕ

1. Словарь «столицы»: страна — столица (5 пар). Выведи столицу Таджикистана.
2. Словарь-меню кафе: блюдо — цена. Посчитай стоимость двух блюд.

★ **Со звёздочкой:** программа спрашивает имя и ищет его телефон в словаре; если нет — говорит «не найдено».

«Крестики-нолики»: строим поле

ЧТО ПОСТРОИМ СЕГОДНЯ

- Игровое поле 3×3 из списка
- Функцию, которая красиво рисует поле
- Ход игрока: ставим крестик в выбранную клетку

Наш большой проект

«Крестики-нолики» на двоих — это проект на два урока. Сегодня строим **поле и ходы**, на следующем уроке добавим проверку победы и защитим проект. Здесь соберётся всё из этой четверти: списки, функции, циклы.

1	2	3
4	X	6
7	0	9

Клетки пронумерованы 1–9. Игрок называет номер — туда ставится его знак.

Поле и его показ

Поле — это список из 9 клеток. Функция рисует его сеткой 3×3:

```
pole.py

board = [" "] * 9    # 9 пустых клеток

def show(board):
    print(board[0], "|", board[1], "|", board[2])
    print("-----")
    print(board[3], "|", board[4], "|", board[5])
    print("-----")
    print(board[6], "|", board[7], "|", board[8])

show(board)
```

`[" "] * 9` создаёт список из девяти пробелов — пустых клеток. Клетка 0 — верхняя левая.

2 Функция `show` рисует поле

Мы вынесли рисование в функцию — теперь можно показать поле в любой момент одной командой `show(board)`.

Теперь ход игрока — ставим знак в выбранную клетку:

```
hod.py

pos = int(input("Куда ставишь (1-9)? "))
board[pos - 1] = "X"  # игрок назвал 1, а список с 0
show(board)
```

ВАЖНЫЙ МОМЕНТ

Игрок называет клетки 1–9, а список считает с 0. Поэтому `board[pos - 1]`: вычитаем 1, чтобы номер клетки совпал с номером в списке.

👉 ЗАДАНИЕ В ПАРЕ

Соберите поле

Наберите поле и функцию `show`. Сделайте два хода вручную: поставьте X в клетку 5 и O в клетку 1. Проверьте, что поле рисуется правильно.

На следующем уроке превратим это в полную игру.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Допиши: пусть программа спрашивает клетку и ставит X, затем показывает поле.
2. Подумай: как чередовать X и O между двумя игроками? Запиши идею словами.

★ **Со звёздочкой:** сделай так, чтобы нельзя было поставить знак в уже занятую клетку.



«Крестики-нолики»: игра и защита

ЧТО ДОДЕЛАЕМ

- Чередование игроков X и O
- Проверку победы функцией
- Готовую игру — и защиту проекта



Собираем игру целиком

Соединяем поле с прошлого урока, чередование игроков и проверку победы:

```
krestiki.py

board = [" "] * 9

def show(board):
    print(board[0], "|", board[1], "|", board[2])
    print(board[3], "|", board[4], "|", board[5])
    print(board[6], "|", board[7], "|", board[8])

def winner(b):
    lines = [(0,1,2),(3,4,5),(6,7,8),
             (0,3,6),(1,4,7),(2,5,8),
             (0,4,8),(2,4,6)]
    for a, b2, c in lines:
        if b[a] == b[b2] == b[c] != " ":
            return b[a]
    return None

player = "X"
for turn in range(9):
    show(board)
    pos = int(input(player + ", клетка (1-9): "))
    board[pos - 1] = player
    if winner(board):
        show(board)
        print(player + " победил!")
        break
    player = "O" if player == "X" else "X"
```

Восемь способов победить: 3 ряда, 3 столбца, 2 диагонали. Функция проверяет каждую линию — три одинаковых знака подряд.

2 Цикл на 9 ходов

Максимум 9 клеток — значит 9 ходов. После каждого проверяем победу.

3 Смена игрока

Последняя строка меняет X на O и обратно: `"0" if player == "X" else "X"`.

ИТОГОВЫЙ ПРОЕКТ ЧЕТВЕРТИ

Доделайте, украсьте, защитите

Соберите игру и сыграйте пара на пару. Затем улучшите: добавьте сообщение о ничьей, запретите ходить в занятую клетку, добавьте рамку. На защите (2 минуты) расскажите: как устроено поле, как работает проверка победы, что было сложно.

Оба в паре должны уметь объяснить функцию `winner`.

ЧТО ПОКАЗАТЬ НА ЗАЩИТЕ

1. Игра работает: двое ходят по очереди, победа определяется верно.
2. Вы добавили хотя бы одно улучшение (ничья, защита клетки, рамка).
3. Каждый может объяснить, как функция `winner` находит победителя.

★ **Со звёздочкой:** добавьте счёт — сколько партий выиграл каждый игрок за вечер.

Графика и большой проект

Черепашка рисует на экране, программы запоминают данные в файлах. А в финале — своя «Змейка».

Четверть 3 · 9 уроков

ЧТО В ЭТОЙ ЧАСТИ

1. **Урок 15.** Черепашка turtle: первые линии
2. **Урок 16.** Циклы и turtle: узоры
3. **Урок 17.** Функции и turtle: рисуем имя
4. **Урок 18.** Файлы: программа запоминает
5. **Урок 19.** 🖨️ Проект «Записная книжка»
6. **Урок 20.** Защита и разбор ошибок
7. **Урок 21.** Готовимся к «Змейке»
8. **Урок 22.** 🐍 Проект «Змейка»: начало
9. **Урок 23.** 🐍 «Змейка»: финал и защита

Черепашка turtle: первые линии

ЧЕМУ НАУЧИМСЯ

- Запускать графику модулем **turtle**
- Двигать черепашку: вперёд, поворот
- Нарисовать квадрат и треугольник



Программа умеет рисовать

До сих пор наши программы работали с текстом и числами. Но Python умеет и **рисовать**. В нём есть встроенный модуль **turtle** — «черепашка». Она ползает по экрану и оставляет линию. Командуешь ей — получаешь рисунок.



На пальцах. Представь черепашку с кисточкой на спине. Куда она ползёт — там остаётся краска. Ты говоришь «вперёд на 100», «повернись направо» — и из этих команд складывается картинка.



Разбираем код

Рисуем квадрат — четыре раза «вперёд и поворот»:

```
kvadrat.py

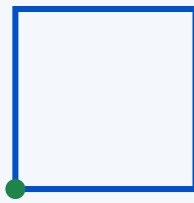
import turtle

t = turtle.Turtle()

t.forward(100)    # вперёд на 100
t.right(90)       # поворот направо на 90°
t.forward(100)
t.right(90)
t.forward(100)
t.right(90)
t.forward(100)

turtle.done()     # оставить окно открытым
```

НА ЭКРАНЕ ПОЯВИТСЯ



Команда `forward` ведёт черепашку вперёд, `right` поворачивает на угол. Четыре стороны по 100 с поворотами на 90° — получается квадрат. `turtle.done()` в конце не даёт окну закрыться сразу.

КОМАНДЫ ЧЕРЕПАШКИ

`forward(n)` / `backward(n)` — вперёд / назад. `right(угол)` / `left(угол)` — поворот.
`t.color("red")` — цвет линии. Углы в градусах: полный круг — 360° .

👉 ЗАДАНИЕ В ПАРЕ

Треугольник

Нарисуйте треугольник. Подсказка: три стороны, а поворачивать нужно на 120° (потому что $360 \div 3 = 120$).

Водитель за клавиатурой, штурман сверяется с учебником. Через 10 минут — меняетесь.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
import turtle
forward(100)
```

окно мгновенно закрылось

А ТАК ПРАВИЛЬНО

```
t = turtle.Turtle()
t.forward(100)
```

`turtle.done()` # в конце

Сначала создай черепашку `t = turtle.Turtle()`, потом командуй ей. И `turtle.done()` в конце, чтобы окно не закрылось.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Нарисуй прямоугольник (стороны разной длины).
2. Нарисуй свои инициалы (буквы из прямых линий: Л, Т, Г).

★ **Со звёздочкой:** нарисуй домик — квадрат и треугольник-крышу.

Циклы и turtle: рисуем узоры

ЧЕМУ НАУЧИМСЯ

- Соединять цикл **for** с черепашкой
- Рисовать фигуры коротким кодом
- Создавать спирали и узоры



Цикл экономит команды

Квадрат на прошлом уроке занял 7 строк. Но ведь мы повторяли одно и то же: «вперёд, поворот». А что повторяется — просится в **цикл**. Соединим то, что уже знаем: `for` из четверти 1 и черепашку.



На пальцах. Вместо того чтобы 4 раза писать «вперёд, направо», говоришь: «повтори 4 раза: вперёд, направо». Короче, понятнее, и легко поменять 4 на 5 — получится пятиугольник.



Разбираем код

Квадрат в цикле — всего 3 строки вместо 7:

```
uzory.py

import turtle
t = turtle.Turtle()

# квадрат в цикле
for i in range(4):
    t.forward(100)
    t.right(90)

turtle.done()
```

Хочешь другую фигуру — поменяй число повторов и угол. Правило простое: **угол = $360 \div \text{число сторон}$** . Для шестиугольника: 6 сторон, поворот 60° .

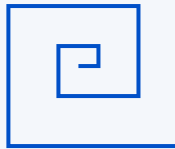
А вот узор-спираль — черепашка рисует и каждый раз чуть-чуть увеличивает шаг и угол:

```
spiral.py

t = turtle.Turtle()
for i in range(100):
```

```
t.forward(i)      # шаг растёт: 0,1,2,3...
t.right(91)       # чуть больше 90 – закрутка
turtle.done()
```

ПРИМЕРНО ТАК



ФОРМУЛА ФИГУРЫ

Правильная фигура: `for i in range(число_сторон)`, внутри `forward(длина)` и `right(360 / число_сторон)`. Меняешь одно число — меняется фигура.

👉 ЗАДАНИЕ В ПАРЕ

Пятиугольник и звезда

Нарисуйте правильный пятиугольник (5 сторон, поворот 72°). Потом попробуйте звезду: 5 повторов, но поворот 144° .

Через 10 минут поменяйтесь ролями.

⚠ Частые ошибки

ТАК НЕ РАБОТАЕТ

```
for i in range(4):
t.forward(100)
t.right(90)
```

А ТАК ПРАВИЛЬНО

```
for i in range(4):
    t.forward(100)
    t.right(90)
```

Обе команды должны быть с отступом — тогда они повторяются вместе. Без отступа цикл «не увидит» вторую строку.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Нарисуй шестиугольник и восьмиугольник (посчитай углы по формуле).
2. Нарисуй ряд из трёх квадратов (подсказка: между ними подними перо `t.penup()`).

★ **Со звёздочкой:** нарисуй «цветок» — повтори круг 12 раз, каждый раз поворачивая на 30° .

Функции и turtle: рисуем имя

ЧЕМУ НАУЧИМСЯ

- Выносить рисование фигуры в функцию
- Управлять пером: поднять и опустить
- Рисовать много фигур в разных местах



Функция-художник

Соединяем ещё две знакомые вещи: **функции** из четверти 2 и черепашку. Опишем «как рисовать квадрат» один раз в функции — и будем рисовать квадраты где угодно одной командой.



На пальцах. Это как штамп. Вырезал штамп «квадрат» один раз — и ставишь его на бумаге сколько хочешь, в любом месте. Функция `kvadrat()` — такой штамп.



Разбираем код

Функция рисует квадрат заданного размера, а перо помогает переходить без линии:

```
imyа.py

import turtle
t = turtle.Turtle()

def kvadrat(size):
    for i in range(4):
        t.forward(size)
        t.right(90)

kvadrat(50)           # маленький

t.penup()             # поднять перо — не рисуем
t.forward(120)
t.pendown()           # опустить — снова рисуем

kvadrat(80)           # большой, в новом месте

turtle.done()
```

`kvadrat(size)` рисует квадрат любого размера — передаём число при вызове.

2 penup / pendown — перо

Поднятое перо не оставляет линию. Так черепашка переходит в новое место, не соединяя фигуры чертой.

УПРАВЛЕНИЕ ПЕРОМ

`penup()` — поднять (двигаемся без линии). `pendown()` — опустить (снова рисуем). Незаменимо, когда рисуешь несколько отдельных фигур: буквы имени, ряд домиков.

👉 ЗАДАНИЕ В ПАРЕ

Три квадрата в ряд

Используя функцию `kvadrat` и перо, нарисуйте три квадрата разного размера в ряд, не соединённые линиями.

Через 10 минут поменяйтесь ролями.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
# перешёл, но забыл penup  
t.forward(120)
```

А ТАК ПРАВИЛЬНО

```
t.penup()  
t.forward(120)  
t.pendown()
```

Если не поднять перо, черепашка нарисует лишнюю линию между фигурами. Не забудь опустить перо обратно.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Напиши функцию `treugolnik(size)` и нарисуй два треугольника рядом.
2. Нарисуй первую букву своего имени из линий и фигур.

★ **Со звёздочкой:** напиши функцию, рисующую квадрат заданного цвета — цвет передавай аргументом.

Файлы: программа запоминает

ЧЕМУ НАУЧИМСЯ

- Сохранять данные в файл .txt
- Читать данные из файла обратно
- Понять, почему это важно для настоящих программ



Проблема забывчивости

Все наши программы **забывали** всё, как только закрывались. Ввёл данные, программа посчитала — закрыл, и всё пропало. Настоящие программы так не делают: заметки, игры, контакты сохраняются. Секрет — в **файлах**.



На пальцах. Память программы — как разговор: закончился и забылся. Файл — как записная книжка: закрыл, убрал, а завтра открыл — всё на месте. Файл переживает закрытие программы.



Записываем в файл

```
zapis.py

# "w" — режим записи (write)
file = open("zametki.txt", "w")
file.write("Купить хлеб\n")
file.write("Позвонить бабушке\n")
file.close()

print("Заметки сохранены!")
```

`open` открывает файл, `"w"` означает «для записи». `write` записывает строку, а `\n` — это переход на новую строку. `close` обязательно закрывает файл — как закрыть кран.



Читаем из файла

```
chtenie.py

# "r" — режим чтения (read)
file = open("zametki.txt", "r")
text = file.read()
```

```
file.close()
```

```
print(text)
```

НА ЭКРАНЕ ПОЯВИТСЯ

Купить хлеб

Позвонить бабушке

ВАЖНО ЗАПОМНИТЬ

`open(имя, "w")` — записать (стирает старое!). `open(имя, "r")` — прочитать. `write` пишет, `read` читает, `close` закрывает. `\n` — новая строка. Есть ещё `"a"` — дописать в конец, не стирая.

👉 ЗАДАНИЕ В ПАРЕ

Дневник

Программа спрашивает, как прошёл день, и сохраняет ответ в файл `dnevnik.txt`. Затем отдельная программа читает файл и показывает запись.

Через 10 минут поменяйтесь ролями.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
file = open("f.txt", "w")
file.write("текст")
# забыли close
```

```
write("привет")\n # это не перенос
```

А ТАК ПРАВИЛЬНО

```
file = open("f.txt", "w")
file.write("текст")
file.close()
```

```
write("привет\n")
```

Всегда закрывай файл через `close`. И помни: режим `"w"` стирает старое содержимое — для дописывания используй `"a"`.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Сохрани в файл список из трёх любимых фильмов, потом прочитай и выведи.
2. Программа дописывает (режим `"a"`) новую строку в файл при каждом запуске.

★ **Со звёздочкой:** счётчик запусков — программа читает число из файла, увеличивает и записывает обратно.



«Записная книжка»

ЧТО ПОСТРОИМ

- Программу, которая хранит контакты между запусками
- Соединим словари, циклы и файлы
- Меню с выбором действия



Первая полезная программа

«Угадай число» и «Виселица» были играми. А это — **полезная** программа, которой можно реально пользоваться: добавлять контакты, искать их, и всё сохраняется в файл. Здесь соберётся много знакомого: словари, циклы, условия, файлы.



Собираем книжку

```
knizhka.py

# читаем контакты из файла в начале
contacts = {}
try:
    file = open("contacts.txt", "r")
    for line in file:
        name, phone = line.strip().split(",")
        contacts[name] = phone
    file.close()
except:
    pass # файла ещё нет — не страшно

while True:
    print("1-добавить 2-найти 3-выход")
    choice = input("Выбор: ")

    if choice == "1":
        name = input("Имя: ")
        phone = input("Телефон: ")
        contacts[name] = phone
    elif choice == "2":
        name = input("Кого ищем? ")
        print(contacts.get(name, "Не найдено"))
    else:
        break
```

```
# сохраняем всё обратно в файл
file = open("contacts.txt", "w")
for name, phone in contacts.items():
    file.write(name + "," + phone + "\n")
file.close()
```

1 Загрузка при старте

В начале читаем файл и складываем контакты в словарь. `split(",")` разделяет строку «имя, телефон» на две части.

2 Меню в цикле

Цикл показывает меню снова и снова, пока не выберешь выход. Знакомый приём из «Угадай число».

3 Сохранение при выходе

Перед выходом записываем весь словарь обратно в файл — чтобы контакты не потерялись.

ЗАДАНИЕ В ПАРЕ

Соберите и проверьте

Наберите программу. Добавьте пару контактов, выйдите, запустите снова — контакты на месте? Значит, файл работает!

Разбирайте по частям: сначала меню, потом файл.

ДОМАШНЕЕ ЗАДАНИЕ

1. Добавь в меню пункт «показать все контакты».
2. Сделай, чтобы при поиске несуществующего имени программа предлагала добавить его.

★ **Со звёздочкой:** добавь удаление контакта по имени.

Защита проекта и разбор ошибок

ЧЕМУ НАУЧИМСЯ

- Читать сообщения об ошибках и понимать их
- Находить и исправлять чужие ошибки
- Рассказывать о своей программе на защите



Ошибка — это подсказка

За три четверти вы много раз видели красные сообщения об ошибках. Пора научиться **читать** их спокойно. Python всегда подсказывает, что не так и в какой строке — надо просто присмотреться.



На пальцах. Ошибка — не «ты плохой программист», а навигатор, который говорит «поверни, ты не туда». Опытные программисты ошибаются постоянно — просто быстро читают подсказку и правят.



Три частые ошибки в лицо

```
oшибki.py

# SyntaxError – забыл двоеточие или скобку
if x > 5
    print("много")

# NameError – имя не создано (опечатка)
print(nmae)

# IndentationError – неправильный отступ
for i in range(3):
print(i)
```

ЧИТАЕМ ОШИБКУ

SyntaxError — синтаксис: забыл двоеточие, скобку, кавычку. **NameError** — имя не существует, чаще опечатка. **IndentationError** — отступы. **TypeError** — смешал текст и число. Последняя строка ошибки — самая важная.



ЗАДАНИЕ В ПАРЕ

Охота на баги

Учитель даст программу с 3–4 ошибками. Ваша задача — запустить, прочитать каждую ошибку, исправить и добиться, чтобы программа заработала. Кто быстрее?

Читайте ошибки вслух друг другу — так быстрее находится причина.



Как защищать проект

Защита — это 2 минуты у проектора. Расскажи по простому плану:

1 Что делает программа

Одним предложением: «это записная книжка, хранит контакты в файле».

2 Как она работает

Покажи главное: «меню в цикле, контакты в словаре, сохранение в файл».

3 Что было сложно

Честно расскажи, где застрял и как решил. Это ценят больше всего.



ДОМАШНЕЕ ЗАДАНИЕ

1. Подготовь защиту «Записной книжки» по плану из трёх пунктов.
2. Найди в своём старом коде любую ошибку, которую делал, и запиши, как её узнать.

★ **Со звёздочкой:** составь свою «шпаргалку ошибок» — 5 самых частых и как их чинить.

Готовимся к «Змейке»: разбор идеи

ЧЕМУ НАУЧИМСЯ

- Разбирать большой проект на части до начала кода
- Понимать, из чего состоит игра «Змейка»
- Планировать — главный навык программиста



Сначала думаем, потом кодим

«Змейка» — самый большой проект года. Бросаться писать код сразу — плохая идея. Сначала **разберём игру на части**: что в ней есть, что за чем происходит. Это называется планированием, и это отличает хорошего программиста.



На пальцах. Строитель не начинает класть кирпичи наугад — сначала чертёж. Наш чертёж «Змейки»: понять, из каких кусочков состоит игра, до того как писать первую строку.



Из чего состоит «Змейка»

Разберём игру на понятные части:

1 Змейка — это список

Тело змейки — список координат. Голова — первый элемент. Двигается — добавляем клетку спереди, убираем сзади.

2 Еда — случайная точка

Яблоко появляется в случайном месте (знакомый `random`). Съела — растёт, появляется новое.

3 Управление — клавиши

`turtle` умеет ловить нажатия стрелок и менять направление движения.

4 Проигрыш — врезалась

Если голова столкнулась со стеной или своим хвостом — игра окончена.

ЧТО МЫ УЖЕ УМЕЕМ ДЛЯ «ЗМЕЙКИ»

Списки (тело змейки), циклы (главный цикл игры), функции (движение, проверки), `random` (еда), условия (столкновения), `turtle` (рисование). Вся игра — это сборка того, что вы уже знаете!

ЗАДАНИЕ В ПАРЕ

Нарисуйте план на бумаге

Не за компьютером, а на листе: нарисуйте, как выглядит игра, и запишите словами, что происходит на каждом шаге (голова двигается → проверяем еду → проверяем столкновение). Это ваш чертёж.

Обсуждайте вслух — так рождается понимание.

ДОМАШНЕЕ ЗАДАНИЕ

1. Запиши по шагам, что происходит за один «тик» игры (одно движение змейки).
2. Подумай: где в «Змейке» пригодится список, а где словарь? Обоснуй.

★ **Со звёздочкой:** придумай, как добавить в игру подсчёт очков, и запиши идею.

 «Змейка»:
начало

ЧТО ПОСТРОИМ СЕГОДНЯ

- Окно игры и змейку из одного сегмента
- Движение по нажатию стрелок
- Основу главного цикла игры

 Собираем каркас

По нашему плану строим постепенно. Сегодня — **окно, змейка и управление**. На следующем уроке добавим еду, рост и проигрыш. Не пугайся длины кода — мы разберём его по кусочкам, и каждый кусочек знакомый.

 Каркас игры

```
import turtle

screen = turtle.Screen()
screen.setup(400, 400)
screen.title("Змейка")

# голова змейки
head = turtle.Turtle()
head.shape("square")
head.color("green")
head.penup()

direction = "stop"

def go_up():
    global direction
    direction = "up"

def go_down():
    global direction
    direction = "down"

# ловим нажатия стрелок
screen.listen()
```

```
screen.onkey(go_up, "Up")
screen.onkey(go_down, "Down")

# главный цикл игры
while True:
    screen.update()
    if direction == "up":
        head.sety(head.ycor() + 20)
    if direction == "down":
        head.sety(head.ycor() - 20)
```

1 Окно и голова

`Screen` создаёт окно игры. Голова — черепашка в форме квадрата. `penup`, чтобы не рисовала линию.

2 Функции направления

Каждая клавиша меняет переменную `direction`. Слово `global` позволяет функции менять переменную снаружи.

3 Главный цикл

`while True` — сердце игры. Каждый повтор: обновить экран и подвинуть голову в текущем направлении.

👉 ЗАДАНИЕ В ПАРЕ

Запустите и подвигайте

Наберите каркас и добавьте функции `go_left` и `go_right` по образцу (меняют `x` через `setx`). Запустите — квадрат должен двигаться стрелками.

На следующем уроке добавим еду и проигрыш.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Допиши движение влево и вправо, чтобы змейка слушалась всех четырёх стрелок.
2. Поменяй цвет и размер окна на свой вкус.

★ **Со звёздочкой:** сделай, чтобы змейка не могла развернуться на 180° (вверх нельзя, если едет вниз).



«Змейка»: финал и защита

ЧТО ДОДЕЛАЕМ

- Еду, которую змейка ест
- Проигрыш при столкновении со стеной
- Готовую игру для портфолио — и защиту



Добавляем еду и проигрыш

К каркасу с прошлого урока добавляем еду и проверку столкновений. Показываю ключевые части — вставьте их в главный цикл:

```
snake_final.py

import random

# создаём еду (яблоко)
food = turtle.Turtle()
food.shape("circle")
food.color("red")
food.penup()
food.goto(0, 100)

score = 0

# --- эти проверки идут ВНУТРИ главного цикла ---

# съели еду?
if head.distance(food) < 20:
    x = random.randint(-180, 180)
    y = random.randint(-180, 180)
    food.goto(x, y)
    score = score + 1
    print("Очки:", score)

# врезались в стену?
if abs(head.xcor()) > 190 or abs(head.ycor()) > 190:
    print("Игра окончена! Очки:", score)
    break
```

Отдельная черепашка в форме круга. `head.distance(food)` измеряет расстояние до неё.

2 Съели — новая еда и очко

Если голова близко к еде — переносим еду в случайное место и добавляем очко.

3 Стена — конец игры

`abs()` берёт число без знака. Ушли за 190 по любой оси — `break`, игра окончена.



ГЛАВНЫЙ ПРОЕКТ ГОДА

Доделайте и защитите «Змейку»

Соберите игру целиком. Сделайте её своей: свой цвет, счёт на экране, ускорение. Это ваш лучший проект — сохраните его в портфолио. На защите (3 минуты) покажите игру в действии и расскажите, как она устроена.

Оба должны уметь объяснить главный цикл и проверку столкновений.



ЧТО ПОКАЗАТЬ НА ЗАЩИТЕ

1. Змейка двигается стрелками, ест еду, растёт счёт.
2. Игра заканчивается при столкновении со стеной.
3. Вы добавили минимум одно своё улучшение.

★ **Со звёздочкой:** сделай, чтобы змейка росла (хвост из сегментов) и проигрывала, врезавшись в себя.

Данные и ТВОЙ проект

Анализ текста, защита от ошибок — и твой собственный проект, который ты придумашь и защитишь сам.

Четверть 4 · 7 уроков + приложения

ЧТО В ЭТОЙ ЧАСТИ

1. **Урок 24.** Анализ текста: считаем слова
2. **Урок 25.** Защита от ошибок: try / except
3. **Урок 26.** Повторение: код-битвы
4. **Урок 27.** Выбираем финальный проект
5. **Уроки 28–30.** 🚀 Финальный проект и защита
6. **Приложения.** Шпаргалка, глоссарий, ответы

Анализ текста: считаем слова

ЧЕМУ НАУЧИМСЯ

- Разбивать текст на слова командой **split**
- Считать количество слов и находить самое частое
- Соединять строки, списки и словари для анализа



Текст как данные

Программы умеют не только хранить текст, но и **анализировать** его: сколько слов, какое встречается чаще всего. Так работают поисковики и умные помощники. У тебя уже есть все инструменты — строки, списки, словари. Осталось соединить.



На пальцах. Представь, что рассыпал текст на отдельные слова-карточки. Теперь можно их посчитать, разложить по стопкам, найти самую большую стопку. Именно это делает анализ текста.



Разбираем код

`split` разбивает строку на список слов:

```
analiz.py

text = "кот пёс кот рыба кот пёс"
words = text.split()      # СПИСОК СЛОВ

print("Всего слов:", len(words))

# считаем каждое слово в словарь
count = {}
for word in words:
    if word in count:
        count[word] = count[word] + 1
    else:
        count[word] = 1

print(count)
```

НА ЭКРАНЕ ПОЯВИТСЯ

Всего слов: 6

```
{'кот': 3, 'пёс': 2, 'рыба': 1}
```

Цикл идёт по словам. Если слово уже в словаре — прибавляем 1, если нет — начинаем с 1. В итоге словарь показывает, сколько раз встретилось каждое слово. «кот» — 3 раза, чемпион.

ВАЖНО ЗАПОМНИТЬ

`split()` — разбивает строку на список слов по пробелам. Приём «считать в словарь» (есть ключ — прибавь, нет — создай) очень частый: так считают что угодно — слова, голоса, товары.

👉 ЗАДАНИЕ В ПАРЕ

Анализатор предложения

Программа спрашивает предложение и выводит: сколько всего слов и сколько среди них разных (подсказка: разных — это `len(count)`).

Через 10 минут поменяйтесь ролями.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
count[word] = count[word] + 1
# для нового слова – ошибка
```

А ТАК ПРАВИЛЬНО

```
if word in count: ...
else: count[word] = 1
```

Нельзя прибавлять к слову, которого ещё нет в словаре. Сначала проверь, есть ли оно.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Найди самое длинное слово в предложении.
2. Посчитай, сколько раз встречается заданная буква во всём тексте.

★ **Со звёздочкой:** найди и выведи самое частое слово в тексте (не только словарь, а именно чемпиона).

Защита от ошибок: try / except

ЧЕМУ НАУЧИМСЯ

- Ловить ошибки, чтобы программа не падала
- Использовать **try** и **except**
- Делать программы надёжными для любого пользователя



Программа не должна падать

Что будет, если в калькуляторе на вопрос «введи число» пользователь напишет «привет»? Программа **рухнет** с ошибкой. Настоящие программы так себя не ведут — они ловят ошибку и вежливо просят повторить. Это делают слова **try** и **except**.



На пальцах. try / except — это страховка. «Попробуй (try) сделать. Если не вышло (except) — вот запасной план». Как страховочный трос у скалолаза: сорвался — не упал, а повис.



Разбираем код

```
zaschita.py

try:
    age = int(input("Сколько тебе лет? "))
    print("Через год будет:", age + 1)
except:
    print("Это не похоже на число!")
```

ЕСЛИ ВВЕСТИ «ПРИВЕТ»

Это не похоже на число!
(программа не рухнула, а вежливо ответила)

Код внутри **try** — то, что **пробуем** сделать. Если там ошибка — Python не падает, а прыгает в **except** и выполняет запасной план. Пользователь видит понятное сообщение, а не красную простыню ошибки.

ВАЖНО ЗАПОМНИТЬ

try: — рискованный код. **except:** — что делать при ошибке. Особенно полезно с **int(input())**, делением (вдруг на ноль) и файлами (вдруг нет файла).

Надёжный калькулятор

Возьмите калькулятор из четверти 1 и защитите его: если ввели не число или делят на ноль — программа не падает, а просит повторить.

Через 10 минут поменяйтесь ролями.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
try:
age = int(input())
except:
```

А ТАК ПРАВИЛЬНО

```
try:
    age = int(input())
except:
```

Код внутри try и except — с отступом, как в if. Не забудь двоеточия после обоих слов.



ДОМАШНЕЕ ЗАДАНИЕ

1. Защити программу, которая читает файл: если файла нет — сообщи об этом вежливо.
2. Сделай ввод возраста, который повторяет вопрос, пока не введут число (try внутри цикла).

★ **Со звёздочкой:** раздели except на два: отдельно для «не число», отдельно для «деление на ноль».

Повторение: код-битвы и турнир

ЧТО СДЕЛАЕМ

- Повторим всё за год в игровой форме
- Проведём код-битвы у проектора
- Подготовимся к финальному проекту



Урок-турнир

Сегодня без новых тем — **закрепляем** всё, что прошли за год. Но не скучным повторением, а битвами: пары соревнуются, кто быстрее и красивее решит задачу. Это и повторение, и подготовка к финальному проекту.



Правила код-битвы

- 1 Учитель даёт задачу**
Короткую, на 5–7 минут: например, «программа выводит все чётные числа от 1 до 50».
- 2 Пары решают на время**
Кто первый получил рабочий код — поднимает руку.
- 3 Показ у проектора**
Две пары показывают решения. Класс голосует: чей код короче и понятнее.



Задачи для битвы

Проверь себя — сможешь решить каждую? Это карта всего, что ты умеешь:



Числа

Сумма цифр введённого числа



Циклы

Вывести таблицу квадратов 1–10



Строки

Проверить, палиндром ли слово



Списки

Найти среднее и максимум списка



Функции

Функция, бросающая кубик



Словари

Посчитать буквы в слове

ТУРНИР КЛАССА

Кубок программиста

Играем 5–6 раундов. За каждую победу пара получает очко. В конце — чемпионы четверти. Проигравших нет: решил задачу — уже молодец.

Меняйтесь, кто за клавиатурой, каждый раунд.

ДОМАШНЕЕ ЗАДАНИЕ

1. Реши три любые задачи из карты выше, которые показались сложными.
2. Начни думать над темой финального проекта — что тебе интересно?

★ **Со звёздочкой:** придумай свою задачу для код-битвы и её решение.

Выбираем финальный проект

ЧТО СДЕЛАЕМ

- Выберем тему своего проекта
- Составим план: что должна уметь программа
- Распределим работу на три урока



Твой собственный проект

Финал года — проект, который ты придумываешь и делаешь **сам** (в паре). Не по образцу, а свой. Три урока на работу, потом защита перед классом. Это то, что покажет: ты умеешь программировать не по шаблону, а решать свою задачу.



Идеи для проекта

Выбери из этих или придумай своё:



Викторина

Вопросы по любимому предмету, подсчёт очков



Игра-бродилка

Текстовый квест с выбором пути



Копилка

Учёт денег: доходы, расходы, файл



Список дел

Задачи с сохранением в файл



Рисунок turtle

Сложная картина или узор-генератор



Своя игра

Развитие «Змейки» или новая аркада



Как спланировать

1

Опиши идею одним предложением

«Викторина по истории Таджикистана на 10 вопросов со счётом».

2

Раздели на части

Что должна уметь программа? Выпиши 3–5 пунктов.

 ЗАДАНИЕ В ПАРЕ**План проекта**

Выберите тему, опишите идею одним предложением и выпишите, что программа должна уметь. Покажите план учителю до начала кодирования.

Спорьте, обсуждайте — хороший план экономит массу времени.

 ДОМАШНЕЕ ЗАДАНИЕ

1. Доработай план: нарисуй на бумаге, как программа будет выглядеть.
2. Выпиши, какие темы года понадобятся (списки? файлы? функции?).

★ **Со звёздочкой:** продумай одну «фишку», которой удивишь класс на защите.



Твой проект и защита

ФИНАЛЬНАЯ ПРЯМАЯ

- Три урока на создание своего проекта
- Защита перед классом — покажи, что умеешь
- Диплом об окончании курса



План трёх уроков

1 Урок 28 — каркас

Собери основу программы: главное меню или главный цикл. Пусть работает хоть как-то, пусть криво — главное запускается.

2 Урок 29 — начинка

Добавь основные функции: подсчёт очков, сохранение в файл, проверки. Программа делает то, что задумано.

3 Урок 30 — блеск и защита

Доведи до ума: сообщения, красота, «фишка». Подготовь и проведи защиту.



Как пройти защиту

Защита — 3 минуты у проектора. План простой:

1 Покажи в действии

Запусти программу, продемонстрируй, что она делает.

2 Объясни устройство

Расскажи главное: какие темы года использовал, как всё работает.

3 Поделись трудностями

Что было сложно, где застрял, как решил. Это ценят больше всего.

КРИТЕРИИ ОЦЕНКИ ПРОЕКТА

✓ Программа запускается без ошибок · ✓ Решает задачу, которую задумал · ✓ Есть комментарии и понятные имена · ✓ Оба в паре объясняют любую строку · ★ Бонус за «фишку» сверх задания.



ФИНАЛЬНЫЙ ПРОЕКТ ГОДА

Сделай то, чем будешь гордиться

Это твоя главная работа за год. Вложись, добавь свою идею, доведи до конца. Через год ты откроешь этот код и удивишься, как много уже умел.

Помогайте друг другу — но каждый должен понимать весь код.

🎓 ТЫ ПРОШЁЛ ВСЁ КУРС!

За год ты прошёл путь от «что такое команда» до собственных программ и игр. Ты умеешь: переменные, условия, циклы, списки, словари, функции, файлы, графику, обработку ошибок. Это настоящий фундамент программиста.

★ **Что дальше:** продолжай! Придумывай проекты, изучай новое. Python — только начало. Ты уже программист.



Шпаргалка по Python

Всё, что прошли за год, на одной странице. Держи под рукой, когда пишешь код.

ВЫВОД И ВВОД

```
# показать на экране
print("Привет")
# спросить (вернёт текст!)
name = input("Имя: ")
# спросить число
n = int(input("Число: "))
```

ПЕРЕМЕННЫЕ И ЧИСЛА

```
age = 15          # число
name = "Али"      # текст
x + y    x - y    # + -
x * y    x / y    # * /
x % y     # остаток
```

УСЛОВИЯ

```
if age >= 18:
    print("взрослый")
elif age >= 12:
    print("подросток")
else:
    print("ребёнок")
# == сравнить, = положить
```

ЦИКЛЫ

```
# повторить 5 раз
for i in range(5):
    print(i)
# пока условие верно
while x < 10:
    x = x + 1
```

СПИСКИ

```
fruits = ["яблоко", "банан"]
fruits[0]      # первый
fruits[-1]     # последний
fruits.append("x") # добавить
len(fruits)    # сколько
```

СЛОВАРИ

```
ages = {"Али": 15, "Мади": 14}
ages["Али"]      # найти
ages["Сафар"] = 16 # добавить
ages.get("X", "нет")
```

СТРОКИ

```
word = "памир"
len(word)      # длина
word[0]        # буква
word.upper()   # ЗАГЛАВНЫЕ
"a" in word    # есть ли
word.split()   # на слова
```

ФУНКЦИИ

```
def summa(a, b):
    return a + b

x = summa(2, 3)    # x = 5
```

ФАЙЛЫ

```
f = open("f.txt", "w")
f.write("текст\n")
f.close()
# чтение: "r", дописать: "a"
```

ЗАЩИТА ОТ ОШИБОК

```
try:
    n = int(input())
except:
    print("не число!")
```



Словарь терминов

Простыми словами — все понятия, которые встретились за год.

Программа	Список команд для компьютера, которые он выполняет по порядку.
Переменная	Подписанный «ящик» для хранения значения. Кладёшь по имени, достаёшь по имени.
Строка	Текст в кавычках. «привет», «Худжанд» — это строки.
Число	То, с чем можно считать. Бывает целым (15) и дробным (3.14).
Условие	Проверка «если ... иначе ...», по которой программа выбирает действие (if / else).
Цикл	Повторение команд: for (по счёту) или while (пока условие верно).
Список	Много значений в одном ящике, пронумерованных с нуля. В квадратных скобках [].
Словарь	Пары «ключ — значение». Ищешь по ключу-имени, а не по номеру. В фигурных скобках { }.
Функция	Своя команда с именем, созданная через def. Описал раз — вызываешь сколько нужно.
Аргумент	Данные, которые передаёшь функции при вызове: summa(2, 3) — тут 2 и 3.
return	Слово, которым функция отдаёт результат наружу, чтобы его можно было использовать.
Файл	Место, где данные хранятся и после закрытия программы. Открывают через open.
Модуль	Набор готовых команд, который подключают словом import. Например, turtle, random.
Отступ	4 пробела перед командой. Показывает, что она внутри if, цикла или функции.
Ошибка (Error)	Сообщение Python о проблеме в коде. Не провал, а подсказка, что и где поправить.



Ответы к заданиям

Подсказки и готовые решения заданий в паре. Загляни сюда, только если застрял — сначала попробуй сам!

ЧАСТЬ 1 — ПЕРВЫЕ ПРОГРАММЫ

Урок 2 · Магази́нный чек

```
price = int(input("Цена: "))
count = int(input("Количество: "))
print("Итого:", price * count, "сомони")
```

Урок 3 · Больше из двух

```
a = int(input("Первое: "))
b = int(input("Второе: "))
if a > b:
    print(a)
else:
    print(b)
```

Урок 5 · Сумма от 1 до 100

```
total = 0
for i in range(1, 101):
    total = total + i
print(total)    # 5050
```

ЧАСТЬ 2 — СТРУКТУРЫ И ФУНКЦИИ

Урок 9 · Магази́нная корзина

```
prices = [12, 45, 8, 30, 15]
print("Всего:", sum(prices))
print("Дороже всего:", max(prices))
```

Урок 11 · Калькулятор на функциях

```
def summa(a, b):
    return a + b
def raznost(a, b):
    return a - b

x = int(input("a: "))
y = int(input("b: "))
print(summa(x, y), raznost(x, y))
```

ЧАСТЬ 3 — ГРАФИКА И ФАЙЛЫ

Урок 15 · Треугольник

```
import turtle
t = turtle.Turtle()
for i in range(3):
    t.forward(100)
    t.left(120)
turtle.done()
```

Урок 16 · Пятиугольник

```
for i in range(5):
    t.forward(80)
    t.right(72)    # 360 / 5 = 72
```

ЧАСТЬ 4 — ДАННЫЕ

Урок 24 · Анализатор предложения

```
text = input("Предложение: ")
words = text.split()
print("Всего слов:", len(words))
print("Разных:", len(set(words)))
```

Урок 25 · Надёжный калькулятор

```
try:
    a = int(input("a: "))
    b = int(input("b: "))
    print(a / b)
except:
    print("Ошибка: проверь ввод!")
```

💡 Остальные задания решаются по этим образцам. Если не получается — вернись к соответствующему уроку и перечитай раздел «Разбираем код».